

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE:ITA0605

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptron's, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 10%

QUESTIONS:15

Total Marks:25

Annexure A

DEBUGGING & OPTIMIZATION TASK QUESTIONS

Code of Conduct:

I Cheredla Jasvina/192324026 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Short Answer Test

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence.

Answer :

1. Possible causes

a) Improper learning rate

- If the learning rate is too large, weights may oscillate instead of settling.
- If it is too small, convergence can be extremely slow.
- Try values such as 0.01, 0.1, and 0.5 and compare the number of updates.

b) Incorrect weight initialization

- Very large or NaN weights can create unstable behavior.
- Initialize weights with small values, commonly zeros or small random values.
- For a standard perceptron, zero initialization is valid and should still converge on a linearly separable dataset.

c) Bias handling errors

The perceptron computes:

- If the bias is omitted, incorrectly initialized, or not updated, the model may fail to find the required separating hyperplane.
- A useful implementation technique is to add a constant feature:
- Then the bias becomes another weight.

d) Incorrect update rule

For prediction:

For a misclassified sample, the update should be:

- A common mistake is updating the weights when the sample is already correctly classified or using the wrong sign in the update.

2. Debugging strategies

1. Verify the dataset

- Check that labels are correctly encoded.
- Confirm that the data is actually linearly separable.
- Check for mislabeled samples or unexpected outliers.

2. Print weights after every update

3. Epoch 1: $w = [...]$, $b = ...$

Epoch 2: $w = [...]$, $b = ...$

If the values explode or continuously oscillate, investigate the learning rate or update rule.

4. Track misclassification count

Record the number of errors in every epoch:

5. Epoch 1 $\rightarrow$ 8 errors

6. Epoch 2 $\rightarrow$ 5 errors

7. Epoch 3 $\rightarrow$ 2 errors

Epoch 4 $\rightarrow$ 0 errors

On a separable dataset, it should eventually reach zero.

8. Test on a tiny dataset

Use a simple AND/OR dataset where the correct decision boundary is known. This makes update-rule errors much easier to identify.

9. Check the prediction function

Make sure the same threshold and bias convention are used during both training and prediction.

10. Check numerical values

Look for NaN, inf, extremely large features, or inconsistent feature scales.

3. Techniques for faster and stable convergence

- Normalize or standardize features so one feature does not dominate the others.
- Use a moderate learning rate, such as 0.01–0.1, and experiment systematically.
- Shuffle the training samples at the beginning of each epoch to avoid unfavorable ordering.
- Initialize weights to zero or small random values.
- Include and correctly update the bias.
- Set a maximum number of epochs and stop early when the number of errors becomes zero.
- Use adaptive learning-rate strategies if the basic implementation is still unstable.
- For very large datasets, use efficient vectorized operations rather than unnecessary loops.

Conclusion :

For a linearly separable dataset, persistent non-convergence strongly suggests a bug in the implementation, incorrect bias handling, unsuitable learning rate, bad data preprocessing, or incorrect labels. The best debugging approach is to first test the perceptron on a tiny known-separable dataset, print every weight update, track errors per epoch, and verify the update equation. Once the implementation is correct, feature normalization, shuffling, and a suitable learning rate can significantly improve convergence speed and stability.

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance. (5 Marks)

Answer :

1. Possible reasons:

- Vanishing gradients: In deep networks, gradients can become extremely small during backpropagation, making earlier layers learn very slowly.
- Inappropriate activation functions: Sigmoid or tanh can saturate and produce very small gradients. ReLU or its variants are usually better for hidden layers.
- Poor weight initialization: Very large or very small initial weights can cause exploding or vanishing gradients. Xavier/Glorot or He initialization can provide better starting weights.
- Suboptimal learning rate: A learning rate that is too small causes slow learning, while one that is too large can cause oscillation or failure to converge.
- Unnormalized input data: Features with very different scales can make optimization inefficient.

2. Step-by-step debugging:

1. Check the training and validation loss to determine whether the network is actually learning.
2. Verify the input data for missing values, incorrect labels, and different feature scales.
3. Normalize or standardize the input features.
4. Inspect the activation functions and replace saturating functions with ReLU/Leaky ReLU where appropriate.
5. Check weight initialization using He or Xavier initialization.

6. Monitor gradients during training to identify vanishing or exploding gradients.
7. Experiment with different learning rates and batch sizes.
8. Verify that the backpropagation and weight-update equations are implemented correctly.

3. Optimization techniques:

- Use adaptive learning-rate methods such as Adam, RMSProp, or AdaGrad.
- Apply feature normalization/standardization.
- Use He initialization with ReLU or Xavier initialization with tanh/sigmoid.
- Use momentum to accelerate learning and reduce oscillations.
- Apply learning-rate scheduling to gradually reduce the learning rate.
- Use batch normalization to stabilize and speed up training.

Thus, systematic monitoring of losses and gradients, combined with proper normalization, initialization, activation functions, and advanced optimizers such as Adam, can significantly improve MLP convergence and training stability.

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed. (5 Marks)

Answer :

1. Possible reasons:

- **Improper chromosome encoding:** The chromosome may not correctly represent the hypotheses in the search space, causing invalid or incomplete solutions.
- **Incorrect fitness function:** If the fitness function does not accurately measure solution quality, the GA may select poor hypotheses.
- **Premature convergence:** Strong selection pressure may cause the population to become similar too quickly and get trapped in a local optimum.
- **Lack of population diversity:** Low mutation rates or a small population can reduce diversity and prevent exploration of new solutions.
- **Randomness:** Different initial populations and random genetic operations can lead to different results in different runs.

2. Systematic debugging:

1. Verify that chromosome encoding and decoding correctly represent valid hypotheses.
2. Test the fitness function manually using known good and bad solutions.
3. Check whether crossover produces valid offspring.
4. Monitor the best fitness, average fitness, and population diversity after every generation.
5. Check the mutation rate and crossover rate.
6. Run the algorithm with fixed random seeds to determine whether randomness is causing inconsistent results.
7. Compare results across multiple runs using average and best fitness.

3. Optimization strategies:

- Increase the mutation rate if diversity is too low; avoid making it excessively high.
- Adjust selection pressure so that good individuals are favored without eliminating diversity too quickly.
- Use elitism to preserve a small number of the best individuals in each generation.
- Increase population size when the search space is large.
- Use suitable crossover methods that preserve valid chromosome structures.
- Apply adaptive mutation rates to encourage exploration during early generations and exploitation later.

Thus, correct chromosome encoding and fitness evaluation should be verified first, followed by monitoring diversity and convergence. Proper mutation tuning, controlled selection pressure, and elitism can improve solution quality, consistency, and convergence speed.

4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, over-parameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability. (5 Marks)

Answer :

A deep neural network combined with genetic programming may suffer from overfitting and high computational cost when the model is too complex, has too many parameters, or is trained with insufficient data.

1. Possible reasons:

- High model complexity: Too many layers, neurons, or genetic-programming expressions can make the model unnecessarily complex.
- Over-parameterization: A large number of parameters allows the model to memorize training data instead of learning general patterns.
- Insufficient training data: A small dataset may not represent all possible patterns, leading to poor performance on unseen data.
- Complex genetic programming search: Evaluating many candidate programs in every generation increases computational cost.

2. Systematic debugging:

1. Compare training and validation accuracy/loss. High training accuracy with poor validation accuracy indicates overfitting.
2. Check the number of layers, neurons, parameters, and GP tree complexity.
3. Verify whether the training dataset is sufficiently large and representative.
4. Use a separate validation and test set to measure generalization.
5. Monitor model performance across generations to identify unnecessary GP complexity.
6. Measure training time and computational resources used by each candidate model.
7. Check whether duplicate or overly complex genetic-programming solutions are being evaluated repeatedly.

3. Optimization techniques:

- Apply L1/L2 regularization to reduce the effect of unnecessary parameters.
- Use dropout to prevent neurons from becoming overly dependent on each other.
- Apply pruning to remove unnecessary neural-network connections or GP branches.
- Use early stopping when validation performance stops improving.
- Reduce the number of layers, neurons, or GP tree depth.
- Use efficient evaluation models, surrogate models, or fitness approximation to reduce the number of expensive candidate evaluations.
- Increase training data through data augmentation when appropriate.

Thus, controlling model complexity, regularizing the network, pruning unnecessary structures, and using efficient genetic-programming evaluation can reduce computational cost while improving generalization, accuracy, and overall model efficiency.

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization. (5 Marks)

Answer :

A decision tree that gives high training accuracy but poor performance on unseen data is usually suffering from overfitting. The tree may have learned noise and specific patterns in the training dataset instead of general patterns.

1. Possible causes:

- Overfitting: The tree learns the training examples too closely.
- Deep tree structure: Too many branches and leaf nodes make the model overly complex.
- Noisy data: Incorrect values, outliers, or mislabeled data can cause the tree to learn irrelevant patterns.
- Irrelevant features: Unnecessary features may create misleading splits.
- Insufficient training data: A small or unrepresentative dataset may not capture the complete problem.

2. Debugging steps:

1. Compare training accuracy with validation/test accuracy to confirm overfitting.
2. Check the tree depth, number of nodes, and number of leaf nodes.
3. Examine the dataset for missing values, outliers, noise, and incorrect labels.
4. Analyze feature importance to identify irrelevant or unnecessary features.
5. Use cross-validation to check whether poor performance occurs consistently across different data splits.
6. Inspect the confusion matrix and other evaluation metrics to identify which classes are being predicted poorly.

3. Optimization methods:

- Apply pruning to remove unnecessary branches and reduce tree complexity.
- Set limits such as `max_depth`, `min_samples_split`, and `min_samples_leaf`.
- Use cross-validation to select the best hyperparameters.
- Perform feature selection to remove irrelevant or redundant features.
- Remove or correct noisy and mislabeled data.
- Use ensemble methods such as Random Forest when a single decision tree is not sufficient.

Thus, controlling tree depth, pruning unnecessary branches, selecting useful features, and validating the model using cross-validation can significantly reduce overfitting and improve generalization on unseen data.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	30	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	10	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand